CYBERSECURITY FOR PROFESSIONALS BOOTCAMP

HANDS-ON LAB

Traffic Flow Analysis

Week 8, Day 1 • Estimated Duration: 3 hours • Individual Lab

Dominic Yeakel

Traffic Flow Analysis

Lab Overview

Week 8 opens with packet and protocol analysis: network sniffing tools, data manipulation, and PCAP file analysis. This lab builds your ability to read raw network traffic the way a SOC analyst does — capturing it, breaking it down by protocol and conversation, spotting a flow that doesn't belong, and writing a detection rule so it gets caught automatically next time.

Learning Objectives

By the end of this lab, you will be able to:

- Capture and read network traffic using standard sniffing tools.
- Analyze a PCAP file by protocol, conversation, and data volume.
- Identify an anomalous traffic flow that deviates from expected baseline behavior.
- Write and test a basic IDS rule to detect that traffic pattern automatically.

Scenario

Meridian Trust Bank's SOC has been relying on ad hoc traffic review whenever something looks wrong. Your team lead wants a repeatable traffic flow analysis process instead. You've been given a PCAP capture from a monitored branch segment containing a mix of normal business traffic and one flow that shouldn't be there — find it, characterize it, and make sure the IDS catches it next time.

Before You Start

> **This lab runs entirely on your own computer**
>
> There is no shared lab server or VM for this lab. Everything — the sample capture, the analysis, and the IDS rule test — runs locally on your own laptop, using two free tools you'll install once before class. Follow the steps below in order and you will not need anything else.

What you'll be given

Your instructor will post these two files to the class shared drive/LMS before the session. Download both into a folder on your computer before you start Part 1.

- Week8_Day1_Sample_Traffic.pcap — the captured branch-segment traffic you'll analyze. This is the file everything in Parts 1–4 is graded against. Do not rename or edit it.
- mtb-week8.rules — a starter rules file (correct syntax, blank rule) you'll fill in for Part 4.

Step 1 — Create your lab folder

Keeping everything in one folder avoids almost every “command not found” or “file not found” problem later. Do this first.

1. Create a new folder on your Desktop named Week8Lab.
2. Move (or download directly into) Week8_Day1_Sample_Traffic.pcap and mtb-week8.rules into that Week8Lab folder.
3. Confirm both files are visible in the folder before continuing. You should see exactly these two files.

Step 2 — Install Wireshark

Wireshark is free and comes from its official site only — never install it from anywhere else.

If you're on Windows

1. Go to wireshark.org/download.html in your browser.
2. Click “Windows x64 Installer” (or “Windows Installer (64-bit)” — wording varies slightly by version). This downloads a file like Wireshark-4.x.x-x64.exe.
3. Double-click the downloaded .exe to start the installer. If Windows shows a “Do you want to allow this app to make changes?” prompt, click Yes. (This requires administrator rights — if you're on a locked-down work laptop and don't have admin rights, tell your instructor before class so you can be set up on a lab VM instead.)
4. Click through the installer using the default options (Next, Next, I Agree, Next).
5. When you reach the component list, leave everything checked and click Next.
6. When you reach the “Install Npcap?” screen, leave it checked and click Next, then Install on the Npcap sub-installer that opens. Npcap is the driver Wireshark needs to capture live traffic — you need it for the optional Part 1 practice capture, but you do NOT need it just to open the provided PCAP file.
7. Finish the Wireshark installer (click Next through any remaining screens, then Finish).
8. Find Wireshark in your Start menu and open it once to confirm it launches — you should see the Wireshark start screen listing your network interfaces.

If you're on macOS

1. Go to wireshark.org/download.html in your browser.
2. Click the macOS download link (a file like Wireshark 4.x.x Intel 64.dmg or Arm 64.dmg — pick the one matching your Mac; Apple Silicon Macs (M1/M2/M3/M4) use Arm 64, older Intel Macs use Intel 64. If you're not sure which chip you have, click the Apple menu > About This Mac and check the Chip line.)
3. Open the downloaded .dmg file, then drag the Wireshark icon into the Applications folder shortcut shown in that window.
4. Open Applications > Wireshark. The first time, macOS will show a Gatekeeper warning (“Wireshark is an app downloaded from the Internet”) — click Open to allow it.
5. During first launch, macOS may ask you to install the ChmodBPF helper (this grants permission to capture live traffic). Follow the prompt and enter your Mac password when asked. As with Npcap on Windows, this is only needed for the optional Part 1 live-capture practice, not for opening the provided PCAP file.
6. Confirm Wireshark opens and shows the start screen listing your network interfaces.

Step 3 — Install Suricata

Suricata is what you'll use in Part 4 to test your detection rule against the sample PCAP.

If you're on Windows

1. Go to suricata.io/download in your browser.
2. Under Windows, click the 64-bit MSI installer link. This downloads a file like Suricata-7.x.x-1-64bit.msi.
3. Double-click the .msi file to start the installer, and click through with the default options (Next, Next, I accept, Install). If prompted by Windows to allow changes, click Yes.
4. If the installer offers to install Npcap, accept it (harmless to install twice if Wireshark already installed it).
5. Leave “Add Suricata to the system PATH” checked if the installer offers that option — this lets you type suricata directly in a command prompt instead of the full folder path.

6. Finish the installer. By default Suricata installs to C:\Program Files\Suricata, and its default configuration file is at C:\Program Files\Suricata\suricata.yaml — you'll use that exact path in Step 4 below.

If you're on macOS

1. Open the Terminal app (Applications > Utilities > Terminal, or search "Terminal" with Spotlight).
2. If you don't already have Homebrew installed, install it by pasting this into Terminal and pressing Return: /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)" — follow any on-screen instructions it prints (it may ask for your Mac password and tell you to run one or two more setup commands afterward; run exactly what it tells you to).
3. Once Homebrew is installed, install Suricata by running: brew install suricata
4. This will take a few minutes. When it finishes, confirm the config file location by running: brew --prefix suricata — the config file will be at [that path]/etc/suricata/suricata.yaml. You'll use that exact path in Step 4 below (the commands in this lab use $(brew --prefix)/etc/suricata/suricata.yaml, which finds it automatically so you don't have to type the full path yourself).

If you're on Linux (Ubuntu/Debian)

1. Open a terminal.
2. Run: sudo apt update
3. Run: sudo apt install -y suricata
4. The default config file will be at /etc/suricata/suricata.yaml — you'll use that exact path in Step 4 below.

Step 4 — Confirm both tools work

Do this before class so any install problem gets caught early, not while the lab is running.

1. Open Wireshark and confirm it launches without an error (you already did this in Step 2, but check again).
2. Open a terminal (Command Prompt or PowerShell on Windows; Terminal on Mac/Linux) and run: suricata -V
3. You should see output like "This is Suricata version 7.x.x RELEASE". If you instead see "command not found" or "suricata is not recognized", see the Troubleshooting box below.

> **TROUBLESHOOTING — "suricata" command not recognized**
>
> Windows: close and reopen your Command Prompt/PowerShell window (PATH changes only take effect in new windows), then try suricata -V again. If it still fails, run the full path instead: "C:\Program Files\Suricata\suricata.exe" -V
>
> Mac: run which suricata to see if it installed. If brew install suricata reported an error, scroll up in Terminal to read the actual error message and share it with your instructor — don't re-run the install blindly.
>
> This is exactly the kind of thing to sort out before class starts, using the class Slack/help channel, so lab time isn't spent troubleshooting installs.

Lab Environment

Component	Role	Notes
Your own computer	Where you'll install and run both tools	Wireshark + Suricata, both free, both installed above

Component	Role	Notes
Sample PCAP File	Captured traffic from a monitored branch segment	Contains one anomalous flow among normal traffic — provided to you, do not recapture
IDS Rule File	Where you'll write and test your detection rule	You run Suricata against the sample PCAP yourself — no separate sensor needed

Scope and Ethics

> **CAUTION**
> Only analyze the provided sample PCAP. If you do the optional Part 1 live-capture practice, capture only your own machine's traffic for a minute or two — never capture on a network you don't own or have explicit permission to monitor.

Tools You'll Use

- tcpdump / Wireshark's own capture engine — command-line and GUI packet capture
- Wireshark — protocol analysis, stream reconstruction, and statistics
- Suricata — writing and testing IDS detection rules

Part 1: Capturing and Reading Traffic

1.1 — Optional practice capture (ungraded)

This step just confirms your capture tools work. It is not graded and has nothing to do with the anomaly you'll hunt for below.

1. Open Wireshark.
2. On the start screen, double-click your active network adapter (usually the one showing a moving traffic graph — Wi-Fi or Ethernet).
3. Let it capture for about 60 seconds while you browse a couple of normal websites.
4. Click the red square Stop button in the toolbar.
5. Go to File > Save As, name it practice.pcap, and save it anywhere — you won't submit this file.

1.2 — Open the provided sample capture

1. In Wireshark, go to File > Open.
2. Browse to your Week8Lab folder and select Week8_Day1_Sample_Traffic.pcap, then click Open.
3. You should see a list of 300+ packets load into the packet list pane. If nothing loads or you get an error, re-download the file — don't try to fix a corrupted download.

1.3 — Protocol hierarchy breakdown

1. With the sample capture open, go to Statistics (top menu bar) > Protocol Hierarchy.
2. A window opens showing a tree of every protocol seen in the capture (for example: eth > ip > tcp > tls, eth > ip > udp > dns) with a percentage and packet/byte count next to each.
3. Write down every protocol you see and its percentage of total traffic — this is your Part 1 deliverable.

UDP (9.2%)

- DNS (9.2%)

TCP (90.8%

- TLS (11%)
- NetBIOS Session Service (NBSS) (3.7%)
- HTTP (0.3%)
- Data (2.1%) (Unusual ports that don’t map to a protocol)

4. Close the Protocol Hierarchy window when you're done (it doesn't update live, so if you apply filters later, reopen it fresh).

1.4 — Top talkers by volume

1. Go to Statistics > Conversations.
2. Click the TCP tab at the top of the window (and separately check the UDP tab for the DNS traffic).
3. Click the Bytes column header once or twice until the list sorts from highest to lowest total bytes — this puts your biggest conversations at the top.
4. Note the top few rows: source address/port, destination address/port, total packets, and total bytes for each. This is your top-talkers list.

TCP

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B
10.50.20.12	51337	192.0.2.44	4444	21	3 kB	21	11	3 kB	10	543 bytes	274.156340	0.2050	105 kbps	21
10.50.20.11	52010	10.50.20.20	445	11	2 kB	19	6	414 bytes	5	1 kB	227.382014	0.1050	31 kbps	112
10.50.20.11	52546	10.50.20.20	445	11	2 kB	15	6	414 bytes	5	1 kB	192.076650	0.1050	31 kbps	112

UDP

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s
10.50.20.11	56201	10.50.20.1	53	2	232 bytes	12	1	92 bytes	1	140 bytes	66.436200	0.0200	36 kbps	5
10.50.20.11	51824	10.50.20.1	53	2	226 bytes	0	1	90 bytes	1	136 bytes	0.000000	0.0200	36 kbps	5
10.50.20.11	52547	10.50.20.1	53	2	226 bytes	8	1	90 bytes	1	136 bytes	45.803763	0.0200	36 kbps	5

5. Also check the Duration column (far right, you may need to scroll) — a conversation with a lot of bytes in a very short duration behaves differently than one with the same bytes spread over minutes.

1.5 — What looks unexpected

1. Compare what you saw in Protocol Hierarchy and Conversations against what you'd expect on a bank branch segment: DNS lookups, encrypted app traffic to a known banking-core server, file-share access to a branch file server.
2. Note anything that doesn't fit that pattern — an unfamiliar destination address, a port you don't recognize, or a protocol that shouldn't be there. You don't need to have the full answer yet; Part 2 walks you through confirming it.

Unfamiliar IP: 192.0.2.44

Notes: Largest transfer of packets and bytes in the entire capture. Only communicates with 10.50.20.12 through TCP.

Part 2: PCAP Analysis and Data Manipulation

2.1 — Filter down to the suspicious conversation

1. Back in the main Wireshark window (not the Conversations popup), click in the display filter bar at the very top (below the toolbar).
2. Using the unfamiliar address or port you flagged in 1.5, type a filter narrowing to just that conversation, for example: ip.addr==<the address you flagged> (replace with the actual address). Press Enter.
3. The packet list should now show only that one conversation — a handshake (SYN, SYN-ACK, ACK), some data packets, then a teardown (FIN/ACK). If you still see unrelated packets, double-check the address you typed.

2.2 — Follow the TCP stream

1. Right-click any one of the filtered packets in the packet list.
2. Choose Follow > TCP Stream from the right-click menu.
3. A new window opens showing the full reassembled conversation as text — traffic from one side in one color, the other side in another color.
4. Read through it. If it's plaintext, you'll be able to read it directly. Note exactly what it contains — this is the “extracted stream data” your Part 2 deliverable asks for.

Contains customer information for export: acct_number, customer_name, balance, ssn_last4

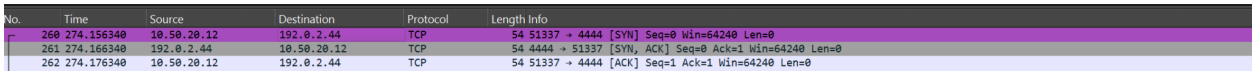
5. Use the “Show and save data as” dropdown in that window to switch to Hex Dump if the content looks binary/garbled instead of readable text, so you can describe what type of data it is even if you can't read it directly.
6. You can copy the visible text directly out of this window (select it, Ctrl+C / Cmd+C) to paste into your lab write-up as evidence. Close the window when done — this also clears the display filter automatically.

MTB_CUSTOMER_EXPORT_BATCH_014
acct_number,customer_name,balance,ssn_last4
40010293,H. Fontaine,48646.87,3854
40010294,K. Kim,19118.57,7897
40010295,F. Duarte,111880.31,8802
40010296,G. Suarez,77044.11,2786
40010297,G. Reyes,33325.88,1710
40010298,F. Lindqvist,68771.41,1510
40010299,R. Kowalski,127113.11,7984
40010300,M. Kowalski,106894.71,8267
40010301,T. Whitfield,48398.12,3430
40010302,T. Okafor,41339.40,9984
40010303,A. Bianchi,149194.81,9692
40010304,D. Osei,75892.00,8048
40010305,M. Duarte,127936.58,4522
40010306,B. Osei,133227.08,7779
40010307,A. Osei,59881.68,2557
40010308,A. Haddad,46961.03,5176
40010309,C. Okafor,103060.74,8770
40010310,T. Whitfield,124785.97,2527
40010311,B. Duarte,111718.99,7105
40010312,S. Nakamura,9109.95,6400
40010313,M. Haddad,53117.28,7929
40010314,R. Lindqvist,90109.46,6761
40010315,M. Fontaine,76170.99,3535
40010316,E. Vasquez,127109.55,6736
40010317,W. Suarez,17339.01,5564
40010318,V. Vasquez,121106.34,1430
40010319,O. Petrova,4459.50,5477
40010320,D. Okonkwo,52731.39,3972
40010321,S. Fontaine,98642.81,2140
40010322,S. Alvarez,13872.64,9691
40010323,P. Bianchi,63044.49,6585
40010324,L. Suarez,46828.21,6314
40010325,V. Abara,12853.36,1861
40010326,S. Marchetti,113194.85,1815
40010327,T. Petrova,66533.77,7947
40010328,E. Abara,66373.58,5475

```
40010329,P. Reyes,17178.97,8043
40010330,M. Lindqvist,101771.30,8972
40010331,F. Lindqvist,6927.15,7475
40010332,O. Kim,1272.14,5938
40010333,P. Okafor,114652.44,5742
40010334,A. Duarte,1278.56,8055
40010335,L. Okafor,57105.49,4770
40010336,F. Nakamura,125023.13,6802
40010337,T. Bianchi,129281.24,1693
40010338,B. Alvarez,69027.97,2275
40010339,A. Fontaine,64461.13,7626
40010340,B. Lindqvist,17382.19,1341
40010341,A. Marchetti,120222.16,8538
40010342,C. Whitfield,65574.36,2734
40010343,H. Haddad,88380.03,9584
40010344,T. Bianchi,130594.20,6562
40010345,H. Okonkwo,116894.19,2251
40010346,F. Duarte,79647.05,4176
40010347,C. Suarez,109139.58,3419
40010348,H. Duarte,22069.06,4232
40010349,L. Abara,23032.12,2234
40010350,L. Kowalski,69657.29,8354
40010351,V. Abara,48562.45,6179
40010352,S. Osei,10354.74,8245
END_TRANSFER
OK
```

2.3 — Characterize the anomaly

1. Using what you saw in the stream and in the Conversations window, note: is the destination address one that appeared anywhere else in the capture? Is the port a standard, well-known port or not? Does the timing/volume look like the other conversations you sorted in 1.4, or different?



No.	Time	Source	Destination	Protocol	Length	Info
260	274.156340	10.50.20.12	192.0.2.44	TCP	54	51337 → 4444 [SYN] Seq=0 Win=64240 Len=0
261	274.166340	192.0.2.44	10.50.20.12	TCP	54	4444 → 51337 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0
262	274.176340	10.50.20.12	192.0.2.44	TCP	54	51337 → 4444 [ACK] Seq=1 Ack=1 Win=64240 Len=0

Source: 10.50.20.12, Port 51337

Destination: 192.0.2.44, Port 4444

2. Write a short paragraph identifying which of those factors (destination, port, timing, volume) make this flow anomalous, referencing the actual numbers you recorded.

The entire export happened in less than a second over ports that have no designated protocol. Each packet sent from 1050.20.12 to 192.0.2.44 is 454 bytes, with the conversation showing plain-text customer information being shared.

2.4 — Document the flow

1. From the Conversations window (Statistics > Conversations, TCP tab), record for this one flow: source IP:port, destination IP:port, protocol, total duration, and total bytes transferred (both directions).
2. This table of values, plus your 2.2 stream findings and 2.3 write-up, is your full Part 2 deliverable.

Source	Src. Port	Destination	Dest. Port	Protocol	Duration	Bytes A-B	Bytes B-A

10.5.20.12	51337	192.0.2.44	4444	TCP	0.205 (s)	3kB	543 bytes

Part 3: Building a Traffic Flow Baseline

1. Go back to Statistics > Conversations with no filter applied (clear the filter bar first, or reopen Wireshark's Conversations window fresh). Look at the TCP and UDP tabs together and summarize: which internal hosts talk to which destinations, over which ports, and roughly how much data each type of conversation moves.
2. Group the conversations into "families" — for example, all the sessions to the same destination on the same port are one family. Based on how many conversations fall into each family and how consistent their size/duration is, describe what a normal flow on this segment looks like.
3. Compare your anomalous flow from Part 2 against that normal pattern, point by point: does it match an existing family, or stand alone? Explain specifically how it deviates (destination, port, volume, timing, or some combination).
4. Rank the anomaly's severity (High/Medium/Low) and justify your rating using the specific evidence from Parts 1–3 — not just a gut feeling.

(Color coded by Port)

Host	Destination	Port	Conversations	Data (out/in)
10.50.20.11	203.0.113.10	443	9	504 B / 690 B
10.50.20.12	203.0.113.10	443	9	504 B / 690 B
10.50.20.11	10.50.20.20	445	3	414 B / ~1 kB
10.50.20.12	10.50.20.20	445	3	414 B / ~1 kB
10.50.20.11	10.50.20.1	53	8	~90 B / ~136 B
10.50.20.12	10.50.20.1	53	7	~90 B / ~136 B
10.50.20.12	203.0.113.5	80	1	466 B / 370 B
10.50.20.12	192.0.2.44	4444	1	~3 kB / 543 B

The red bar is anomalous and I rank the severity at high. The destination IP and port are unique and this conversation has the largest data transfer with 3kB.

Part 4: Writing and Testing an IDS Rule

4.1 — Anatomy of a Suricata rule

Every Suricata rule follows the same shape. Understanding each piece makes writing your own rule much easier than guessing at syntax:

```
action proto  src_ip   src_port  ->  dst_ip     dst_port  (options;)
alert  tcp    $HOME_NET any      ->  !$HOME_NET 4444      (msg:"..."; sid:1000001;
rev:1;)
```

- action — what to do when this rule matches. alert is what you'll use — it logs the match without blocking anything.

- proto — the protocol to match (tcp, udp, http, etc.).
- src_ip / src_port → dst_ip / dst_port — the direction and addresses/ports to match. $HOME_NET is a variable meaning “your internal network” (already defined in Suricata's default config); !$HOME_NET means “anything NOT in your internal network.” any matches any port.
- options (inside the parentheses) — msg is the human-readable alert text; sid is a unique rule ID number you choose (anything not already used); rev is a revision number, starting at 1; content matches specific bytes/text inside the packet payload; flow:to_server,established restricts the match to established connections going to the server side.

4.2 — Write your rule

1. Open mtb-week8.rules from your Week8Lab folder in a plain text editor (Notepad on Windows, TextEdit on Mac — in TextEdit, make sure Format > Make Plain Text is selected so it doesn't save as rich text).
2. Using the anatomy above and the specific IP/port/pattern you documented in Part 2, write one alert rule matching your anomalous flow. Give it sid:1000001; rev:1; and a clear msg describing what it detects.
3. Save the file, keeping it named mtb-week8.rules and keeping it in your Week8Lab folder.

alert tcp 10.50.20.12 any -> 192.0.2.44 4444 (msg:"MTB LAB - Data exfiltration from 10.50.20.12 to 192.0.2.44 on TCP 4444"; flow:to_server,established; dsize:>0; classtype:trojan-activity; sid:1000001; rev:1;)

4.3 — Run your rule against the sample PCAP

Open a terminal (Command Prompt/PowerShell on Windows, Terminal on Mac/Linux) and navigate into your lab folder first:

```
cd Desktop/Week8Lab
mkdir logs
```

Then run Suricata against the sample PCAP using your rule file. Use the command for your operating system:

Windows (Command Prompt or PowerShell)

```
"C:\Program Files\Suricata\suricata.exe" ^
  -c "C:\Program Files\Suricata\suricata.yaml" ^
  -r Week8_Day1_Sample_Traffic.pcap ^
  -S mtb-week8.rules -l logs -k none
```

macOS (Terminal, installed via Homebrew)

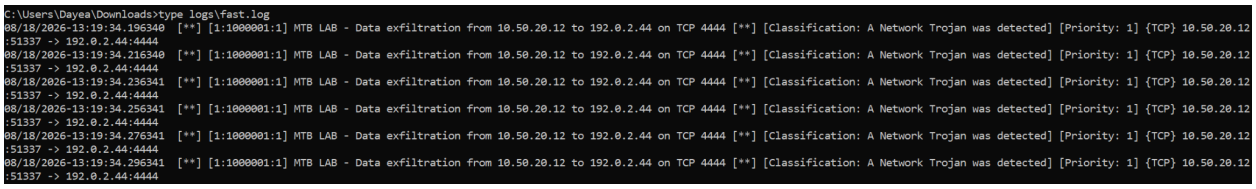
```
suricata -c $(brew --prefix)/etc/suricata/suricata.yaml \
  -r Week8_Day1_Sample_Traffic.pcap \
  -S mtb-week8.rules -l logs -k none
```

Linux

```
suricata -c /etc/suricata/suricata.yaml \
  -r Week8_Day1_Sample_Traffic.pcap \
  -S mtb-week8.rules -l logs -k none
```

1. Let it run — it finishes in under a second for a file this size and returns you to the command prompt.
2. Open the logs folder that was created inside Week8Lab and open fast.log in a text editor (or run cat logs/fast.log on Mac/Linux, or type logs\fast.log on Windows).

3. Each line is one alert: a timestamp, your rule's sid and rev in brackets, your msg text, a classification and priority, and the source/destination IP:port that triggered it. If the file is empty, your rule didn't match — see Troubleshooting below.



4.4 — Confirm no false positives

1. In that same fast.log, check every line's source/destination — every single one should point to the same anomalous flow you documented in Part 2.
2. If you see alerts referencing your normal traffic (the banking-core, file-share, or DNS conversations from Part 1), your rule is too broad — narrow the IP or port match and re-run the command from 4.3.
3. It's normal to see the same conversation logged more than once (Suricata can log an alert per matching packet in that flow, not just once per connection) — that's still one true positive, not multiple false positives.

> **TROUBLESHOOTING — Part 4**
>
> "No such file or directory" for the PCAP or rules file: you're not in the Week8Lab folder. Run cd Desktop/Week8Lab again and confirm with dir (Windows) or ls (Mac/Linux) that both files are listed.
>
> fast.log is empty / your rule never fires: double-check the IP address and port in your rule exactly match what you documented in Part 2 (typos are the most common cause), and make sure the rule line ends with a semicolon after rev:1.
>
> "error parsing signature" when Suricata starts: a syntax mistake in your rule — compare it line-by-line against the anatomy example in 4.1, especially matching parentheses and every option ending in a semicolon.
>
> You can re-edit mtb-week8.rules and re-run the exact same command from 4.3 as many times as you need — there's no setup to redo between attempts.

1. Document your final rule and the alert output it produced — this, plus your 4.4 false-positive check, is your Part 4 deliverable.

Deliverables

Submit the following through the class shared drive/LMS:

- Your Part 1 protocol hierarchy and top-talkers summary
- Your Part 2 anomalous flow documentation, including any extracted stream data
- Your Part 3 traffic flow baseline and anomaly comparison with severity rating
- Your Part 4 IDS rule and evidence it correctly fires on the anomaly without flagging normal traffic

Reflection Questions

Answer briefly in your own words (3–5 sentences each):

1. What specific detail made the anomalous flow stand out once you had a baseline to compare it against?

Every other TCP conversation on the segment followed the same template: 11 packets, about 0.1 seconds, a well-known port, and more data coming back from the server than going out. The flow from 10.50.20.12 to 192.0.2.44 on port 4444 broke that template: it had 21 packets, lasted twice as long, and sent roughly five times more data out than it received. It was also the only conversation to that

destination, and it used a port associated with Metasploit rather than a standard service. On its own, 3 kB of traffic isn't alarming, but next to a baseline of identical, download-leaning sessions, the reversed direction and unique destination made it obvious.

2. Why is establishing a traffic baseline necessary before you can reliably spot an anomaly, rather than just looking for traffic that “looks bad”?

What counts as "bad" depends on the environment. Large outbound transfers are normal for a backup server but suspicious for a workstation, and an unusual port might be a legitimate internal application. Without a baseline, you're relying on guesses about what looks suspicious, which leads both to chasing harmless traffic and to missing attacks that use ordinary-looking ports or protocols. A baseline defines what normal means for this specific network, such as typical destinations, ports, sizes, and directions, so anomalies become measurable deviations rather than impressions.

3. Why does confirming your IDS rule doesn't fire on legitimate traffic matter as much as confirming it fires on the real threat?

A rule that fires on legitimate traffic buries real alerts under noise, and analysts who see constant false alarms learn to ignore or silence the rule, known as alert fatigue. False positives also waste investigation time and can trigger disruptive responses, such as blocking a legitimate service. Testing against the baseline traffic confirms that the rule is precise. Here, it should alert only on the port 4444 flow and stay silent on the 18 HTTPS, 6 SMB, 15 DNS, and 1 HTTP conversations.

Grading Alignment

This lab counts toward your lab-completion and documentation-quality grading criteria for the bootcamp. A detection rule that fires only on the exact sample you were given isn't enough — your documentation should show you understand why the flow was anomalous, not just that you found it once.